

MEDIATEK

MT8676 Yocto Property User Manual

Version: 1.0
Release date: 2025-01-22

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2025-01-22	Min Kuang	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

1 Yocto Property 4

1.1 Functionality of Yocto Property 4

1.1.1 Provide the build.prop and local.prop 4

1.1.2 Support setprop/getprop Command 4

1.1.3 Support Persist Property 4

1.1.4 The API Supported by libprop.so 5

1.1.5 Support Property Watch 5

1.1.6 Specific Property Cross-OS Synchronization 5

1.2 Yocto Property Framework 6

1.3 Property Source Code 6

1.4 How to Use libprop.so 6

1.5 How to Add a Property to local.prop 7

1.6 Property Watch 7

1.7 Specific Property Cross-OS Synchronization 8

Exhibit 1 Terms and Conditions 10

List of Figures

Figure 1-1. getprop 4

Figure 1-2. Property API 5

Figure 1-3. Property flow 6

Figure 1-4. prop.bb 7

Figure 1-5. watchprop 7

Figure 1-6. watchprop API 8

Figure 1-7. Property cross-OS synchronization flow 8

Figure 1-8. setprop from Android to Yocto 9

Figure 1-9. setprop from Yocto to Android 9

1 Yocto Property

1.1 Functionality of Yocto Property

1.1.1 Provide the build.prop and local.prop

```
sh-3.2# getprop
[ro.mediatek.version.branch]: [master-spm]
[ro.mediatek.version.release]: [master-spm-2024_05_11_03]
[ro.vendor.mediatek.version.branch]: [master-spm]
[ro.vendor.mediatek.version.release]: [master-spm-2024_05_11_03]
[ro.build.id]: [2024_05_11_03]
[ro.product.device]: [auto8678p1_64_bsp_vm]
[ro.product.model]: [auto8678p1_64_kde_hyp]
[ro.hardware]: [mt8678]
[ro.board.platform]: [mt8678]
[ro.product.name]: [auto8678p1_64_kde_hyp]
[ro.build.flavor]: [auto8678p1_64_kde_hyp]
[ro.build.product]: [auto8678p1_64_kde_hyp]
[ro.build.project]: [auto8678p1_64_kde_hyp]
[ro.build.target]: [auto8678p1_64_kde_hyp]
[ro.mediatek.os]: [yocto]
[ro.build.type]: [userdebug]
[ro.vendor.mbrain.mode]: [mtk]
[vendor.mbrain.ready]: [1]
[ro.kernel.version]: [6.6]
[ro.build.version.release_or_preview_display]: [14]
[sys.boot_completed]: [1]
[ro.build.version.release]: [13]
[ro.vendor.hypervisor]: [1]
```

Figure 1-1. getprop

1.1.2 Support setprop/getprop Command

For example: setprop xxx xxx

getprop xxx or getprop | grep xxx (xxx refers to property key and value)

1.1.3 Support Persist Property

For example: setprop persist.xxx xxx

getprop persist.xxx
cat data/property/xxx

1.1.4 The API Supported by libprop.so

```
int property_get(const char *key, char *value, const char *default_value);
int8_t property_get_bool(const char *key, int8_t default_value);
int64_t property_get_int64(const char *key, int64_t default_value);
int32_t property_get_int32(const char *key, int32_t default_value);

/* property_set: returns 0 on success, < 0 on failure
 */
int property_set(const char *key, const char *value);

#ifdef MTK_BINDER_NOSUPPORT
int property_watch(const char *key, callback_t callback);
int property_watch_async(const char *key, callback_t_async callback);
/* system=0 for vendor property, system=1 for system property
** boot=0 for no wait android ready, boot=1 for wait android ready
 */
int property_set_sync_android(const char *key, const char *in_value, int system, int boot);
int property_set_only_android(const char* key, const char* value, int system);
int property_set_sync_stou(const char *key, const char *value, int system);
int property_set_only_uos(const char* key, const char* value);
#endif

int property_list(void (*propfn)(const char *key, const char *value, void *cookie), void *cookie);
```

Figure 1-2. Property API

1.1.5 Support Property Watch

See [Section 1.6](#).

1.1.6 Specific Property Cross-OS Synchronization

See [Section 1.7](#).

1.2 Yocto Property Framework

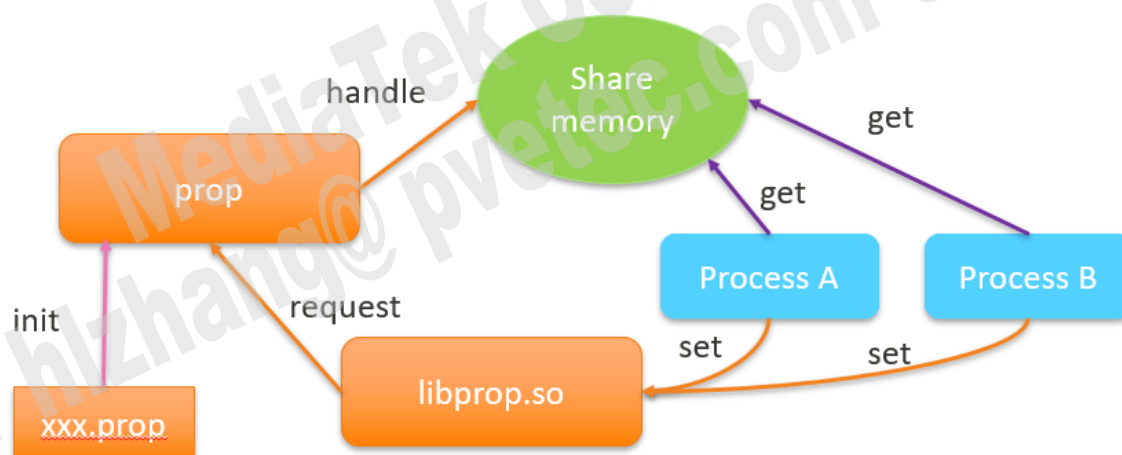


Figure 1-3. Property flow

The Yocto property includes a prop daemon service and a libprop.so. Modules that link to libprop.so can invoke its API. The following steps describe the communication process:

1. When a process calls property set, communication occurs with prop.
2. Prop writes the property to shared memory.
3. When a process calls property get, it directly reads from shared memory.

1.3 Property Source Code

Yocto:

- source path: `src/apps/atom-base/progs/property/`
- bb path:
`meta/meta-mediatek/recipes-devtool/prop/prop.bb`
`meta/meta-mediatek/recipes-devtool/prop/propsync.bb`
`meta/meta-mediatek/recipes-devtool/libprop/ libprop.bb`

Android:

- Source path: `vendor/mediatek/proprietary/external/propsync/`

1.4 How to Use libprop.so

If a module wants to call the property API, it needs to link libprop.so, the steps are as follows:

1. Modify the bb file

```
DEPENDS = "libprop"
```

2. Add to the makefile

LDFLAGS: -lprop

3. Include properties.h in the source code

```
#include <prop/properties.h>
```

4. xxx.service depends on prop.service (especially since it starts earlier than prop during boot, it is necessary to wait until prop is ready before use.)

After=prop.service

1.5 How to Add a Property to local.prop

In `prop.bb`, place the properties that need to be generated during build time in `local.prop`.

```
do_compile:append() {
    » if [ -f ${VISION_SRC} ]; then
    »     buildtime="$(cat ${VISION_SRC} | cut -d':' -f 2)"
    »     buildbranch="$(cat ${VISION_SRC} | cut -d'/' -f 1)"
    »     buildproject="$(cat ${VISION_SRC} | cut -d'/' -f 2)"
    »     buildtarget="$(cat ${VISION_SRC} | cut -d'/' -f 2)"
    »     echo '[ro.mediatek.version.branch]: ['$buildbranch']' >> ${S}/local.prop
    »     echo '[ro.build.id]: ['$buildtime']' >> ${S}/local.prop
    »     echo '[ro.product.device]: ['${MTK_PROJECT}']' >> ${S}/local.prop
    »     echo '[ro.hardware]: ['${TARGET_PLATFORM}']' >> ${S}/local.prop
    »     echo '[ro.build.type]: ['${MTK_PROJECT}']' >> ${S}/local.prop
    »     echo '[ro.build.project]: ['$buildproject']' >> ${S}/local.prop
    »     echo '[ro.build.target]: ['$buildtarget']' >> ${S}/local.prop
    » fi
}
```

Figure 1-4. `prop.bb`

1.6 Property Watch

- The `property_watch()` function is developed to monitor a specific key. When this key is set, it notifies the process that calls `property_watch` to execute the callback function.

UT test method: On the Yocto side, execute `watchprop -w xxx(key) xxx(value)`

```
sh-3.2# watchprop -w km.test test
watchprop callback key=km.test, value=testsh-3.2#
```

Figure 1-5. `watchprop`

- If a module wants to monitor a specific property, call the `property_watch` function directly and implement a custom callback function.

Note: Property watch includes synchronous callback and asynchronous callback.

If the return value of the callback is not a concern or if the callback implementation involves blocking and time-consuming actions, use asynchronous callbacks by calling `property_watch_async()`.

To obtain the callback return value through the `property_set` function, please use `property_watch()`.

```

39 int property_watch(const char *key, callback_t callback);
40 int property_watch_async(const char *key, callback_t_async callback);

26 typedef int (*callback_t)(const char *key, const char *value);
27 typedef void (*callback_t_async)(const char *key, const char *value);
28

```

Figure 1-6. watchprop API

1.7 Specific Property Cross-OS Synchronization

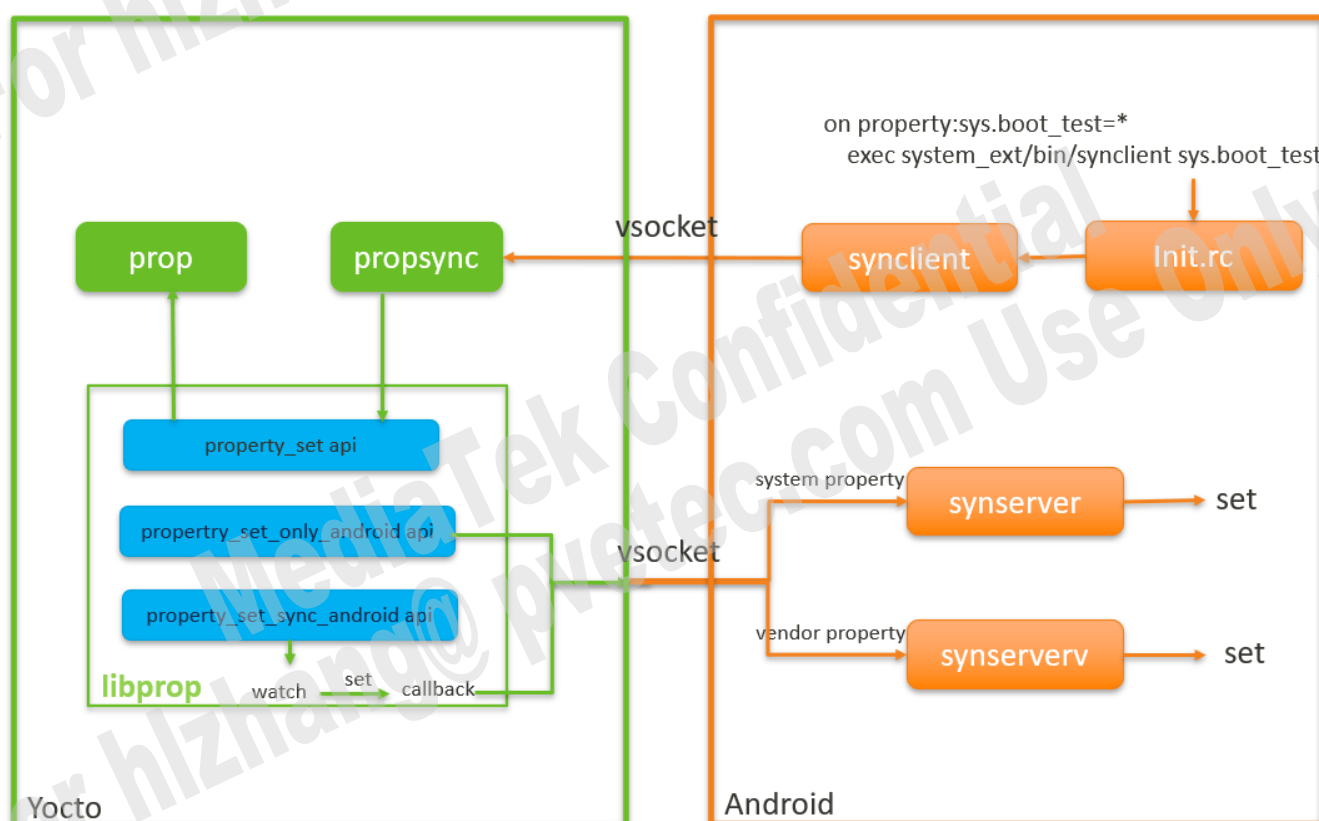


Figure 1-7. Property cross-OS synchronization flow

1. Android synchronization with Yocto (in the Android rc file, set the property to be monitored. When this property is set, it will trigger the process “synclient” to run. The synclient will communicate with the Yocto propsyncservice through vsocket. Then, the Yocto propsyncserver will set the property.)

UT test:

- 1) Monitor sys.boot_test in init.project.rc

```

71 on property:sys.boot_test=*
72   exec system_ext/bin/synclient sys.boot_test

```

- 2) On the Android side: `adb shell setprop sys.boot_test done`
- 3) On the Yocto side check: `adb shell getprop sys.boot_test`

```

km2
auto8678p1_64_bsp_vm:/ # setprop sys.boot_test done
auto8678p1_64_bsp_vm:/ #
sh-3.2# getprop sys.boot_test
done
sh-3.2#

```

Figure 1-8. setprop from Android to Yocto

- Yocto synchronization with Android (Call the `property_set_sync_android` function directly. The callback function communicates with the Android synserver or synserverv through a vsocket. Subsequently, synserver or synserverv sets this property.)

UT test:

- On the Yocto side:

```

adb shell watchprop -s xxx(key) xxx(value) 0 0 (vendorproperty)
adb shell watchprop -s xxx(key) xxx(value) 1 0 (system property)

```

- Check on Android: `adb shell getprop xxx`

```

sh-3.2# watchprop -s vendor.kml kml 0 1
Client connected to cid 3 port 4005
Message sent to server
Received response from server: This is a custom string from Android server.
sh-3.2# watchprop -s system.kml kml 1 1
Usage: watchprop -w [-s <key> <value>]
^C
sh-3.2# watchprop -s system.kml kml 1 1
Client connected to cid 3 port 4004
Message sent to server
Received response from server: This is a custom string from Android server.
sh-3.2# watchprop -s system.kml kml 1 0
Client connected to cid 3 port 4004
Message sent to server
Received response from server: This is a custom string from Android server.
sh-3.2#

```

```

[ro.vendor.dlkm.build.date.utc]: [1719332405]
[ro.vendor.dlkm.build.fingerprint]: [alps/vext_auto8678p1_64_bsp_vm/auto8678p1_64_bsp_vm
fp79:userdebug/dev-keys]
[ro.vendor.dlkm.build.id]: [001A.240205.004]
[ro.vendor.dlkm.build.tags]: [dev-keys]
[ro.vendor.dlkm.build.type]: [userdebug]
[ro.vendor.dlkm.build.version.incremental]: [alliberautoofp79]
[ro.vendor.dlkm.build.version.release]: [14]
[ro.vendor.dlkm.build.version.release.ori.codename]: [14]
[ro.vendor.dlkm.build.version.sdk]: [34]
[system.kml]: [kml]
[system.kml]: [kml]
[vendor.kml]: [kml]
[vendor.kml]: [kml]
auto8678p1_64_bsp_vm:/ #
C:\Users\muk07932>

```

Figure 1-9. setprop from Yocto to Android

Note: Please turn off selinux before testing (`adb shell setenforce 0`)

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.